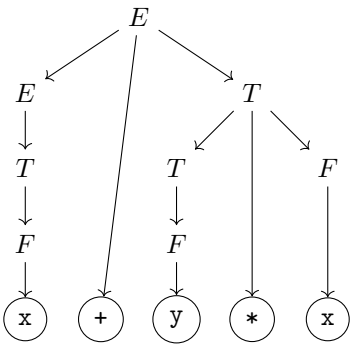
第5章　意味解析

構文解析で得られた構文木はプログラムの構造を形式的に表現している．構文木のノードにはプログラムの意味に関わる情報が含まれている．例えば，x+y*x という式の構文木は以下のようになる．



ここで，終端記号は文法上では，トークンとして扱われていたが，実際には，変数，演算子などプログラム実行上の意味を持つ．さらに，トークンが同じ変数である場合（例えば，x) は，同じ式の中では割り当てられた同じ値を持つ．一般に+は加算，*は乗算を表す．プログラミング言語の多くは値に「型」という概念を持つ．これは，計算を実行するときにどれだけ記憶領域を確保すればよいかを示すとともに，式やプログラムの操作の記述に不整合がないかを実行前にチェックするために用いられる．プログラムが定める意味に従った構文の整合性を確認することを**意味解析**と呼ぶ．

5.1　記号表

記号表は構文木のノードが表す記号が持つ意味的情報を格納する．構文木において，終端記号としてのトークンは記号情報を持つ．字句解析によって得られたトークンは，識別子 (identifier)，演算子 (operator)，区切り記号 (delimiter) などの種類を持つ．それぞれのトークンに応じてプログラム中での意味が与えられる．記号表は構文に現れる記号の情報を格納する．したがって，記号表の構成はプログラミング言語の定義に依存する．

多くのプログラミング言語では，同じ識別子を異なった文脈で使用することができる．プログラミング言語においてはスコープルールが定義されている．最も基本的なスコープルールは局所と大域に対するルールであり，局所記号は一定のブロック内のみに有効な記号であり，大域記号はプログラム中のすべての場所に有効な記号である．スコープルールとしてはまず局所に有効な記号が存在するかどうかを調べて，局所になければ大域記号を参照するという規則である．

```
1  int x;
2  int y;
3  int f(int y) {
4      x = 2;
5      return x+y;
6  }

7  int main() {
8      int x;
9      x = 1;
10     y = 3;
11     x = f(x);
12     printf("%d,%d\n",x,y);
13 }
```

図 5.1: 局所変数と大域変数の例

例えば上記のプログラムにおいて，3 行目から 6 行目において y は引数からの局所変数であり，7 行目から 13 行目において，x も局所変数となる．その他の x および y は大域変数として参照される．したがって，このプログラムが C 言語と同様に main から実行されるとすると，9 行目は，8 行目で宣言した局所変数への代入であり，10 行目は 2 行目で宣言された y を参照する．関数の引数は 1 であり，この結果として，関数 f は 3 を返す．よって 13 行目の結果として，3,3 が表示される．

コンパイラが実行コードを生成する場合，また，インタプリタがプログラムを実行する際には，適切なメモリ領域を参照する必要がある．大域変数はプログラムの実行前に，局所変数は関数の呼出しごとにメモリ領域を確保してその番地が決定する．記号表は変数（記号）とメモリ番地の対応を管理する．

コンパイラが図 5.1 からコード生成を行う場合，２行目において大域変数 x，y を割り当て，３行目で関数記号 f，局所変数 y を割り当てる．４，５行目の x は１行目で割り当てた大域変数 x を参照し，５行目の y は３行目で割り当てた局所変数 y を参照する．6 行目で y の参照を破棄する．７行目で関数記号 main を割り当て，８行目で局所変数 x を割り当てる．９行目の x は８行目で割り当てた局所変数 x を参照し，10 行目の y は２行目で割り当てた大域変数 y を参照する．11 行目で局所変数 x に関数 f を呼び出し，その結果を局所変数 x に代入する．12 行目における x は局所変数，y は大域変数を参照する．このような参照を可能にするために，変数記号から記号表のどのエントリを参照するかを決める必要がある．図 5.1 において，x と y はともに大域変数の場合と局所変数の場合がある．各変数は宣言された時点で記号表に登録され，局所変数は，有効なブロックの中でのみ有効であり，有効なブロックから出ると参照されることがないので，記号表から削除する．大域変数はプログラムの最初から最後まで有効なので記号表から削除されることはない．

構文の構造に従って参照を決めるスコープルールを**静的スコープ**，呼び出しによって直前の局所変数を参照するスコープルールを**動的スコープ**と呼ぶ．上にあげた参照ルールは静的スコープであ

り，図 5.1 が静的スコープを持つとすると最も最後に追加された記号を参照することによって正しい参照を実現することができる．静的スコープは，構文解析の段階で決定されるため，コンパイル時に参照を決定することができる．一方で，動的スコープは，プログラムの実行時に参照を決定するため，実行時に参照を決定する必要がある．プログラミング言語としては静的スコープがよく用いられるが，動的スコープを持つ言語も存在する．例えば，Lisp 言語は動的スコープを持つ．

プログラミング言語によっては，関数やメソッドのスコープルールが決められている場合がある．例えば，Java 言語において，private なメソッドはそのクラス内でのも参照できるメソッド (操作) であり，オブジェクト指向言語においてよく用いられる．オブジェクト指向言語では，クラスから生成されるインスタンスに固有の局所変数を用いてオブジェクトの独立性を保つことが多く，スコープを指定するディレクティブが多く用意されていることが多い．

5.1.1 記号表の実現

記号表は，構文要素の記号をもとにスコープルールを守りながら，その構文要素が持つ意味的な情報を管理するデータ構造である．例えば，図 5.1 のプログラムでは，9 行目のコンパイルにおいては以下のような属性をもつ表構造として実現される．

	名前	種類	値	スコープ	番地	型	引数個数	引数型
1	`x`	var		大域 (G)	0	`int`		
2	`y`	var		大域 (G)	1	`int`		
3	`f`	func		大域 (G)	100	`int`	1	0
4	`x`	var		局所 (L)	0	`int`		

値フィールドは，定数記号の宣言時に用いられることを想定している．

記号表はコンパイルの過程でスコープルールに従って，記号の登録，参照および削除を行ってプログラムで決められた変数や関数記号が実際にメモリ上のどの領域を意味するか，どの命令系列に対応する関数か，ということを表として管理する．ブロック構造によって文脈が変化する場合，ブロック内で局所変数が宣言された場合は記号表に追加し，ブロックから脱出する場合は局所変数を記号表から削除する．このため，記号表は柔軟に登録，削除ができ，高速に参照できるデータ構造である必要がある．記号データを一列に並べた線形リストは，参照は線形時間時間で可能であるが，スコープルールを実現するためのデータの追加，削除も線形で可能であるが，データ量が大きくなると実際の操作時間は増大する．バランス木などの木構造によって整列した場合は参照は log 時間で可能になり，高速にアクセス可能であるが，追加・削除における木構造の操作に時間がかかる．この目的のため，次に述べるハッシュによるインデックス付けの手法がよく用いられる．

例えば以下のような C 言語風のプログラミング言語において，各関数が呼び出された直後と各関数の環境を出る直前の記号表の様子を示す．番地については，関数についてはコンパイルされた命令の最初の位置，帯域変数では，プロセス内で割り当てられた帯域変数領域の先頭からの相対位置 $(G + n)$，局所変数，仮引数については，スタックフレームにおける局所変数領域の先頭からの相対位置 $(FP + n)$ が記入されている．

```
int g;

func f(int a, real b) {
    int x;
    x = a + 1;
    g = x;
}

func h() {
    int x;
    g = 0;
}

func main() {
    int x;
    x = 3;
    f(x + 1, (2.5 + 3.5) * 2);
    h();
}
```

関数 f の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	—	—

関数 f の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
a	仮引数	呼出し時に決定	local	FP+0	int	—	—
b	仮引数	呼出し時に決定	local	FP+8	real	—	—
x	変数	未定	local	FP+16	int	—	—

関数 h の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	—	—

関数 h の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
x	変数	未定	local	FP+0	int	—	—

関数 main の局所文脈に入るとき（仮引数・局所変数の登録前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
main	関数	—	global	L_main	void	—	—

関数 main の局所文脈を出るとき（局所項目の消去直前）

名前	種類	値	スコープ	番地	型	引数個数	引数型
g	変数	未定	global	G+0	int	—	—
f	関数	—	global	L_f	void	2	(int, real)
h	関数	—	global	L_h	void	0	()
main	関数	—	global	L_main	void	0	()
x	変数	未定	local	FP+0	int	—	—

図 5.2: 記号表の例

ハッシュ

ハッシュは線形に並べたデータをインデックス（目次）ごとに並べるようにして，データ構造の途中から探索する方法である．追加・削除も途中から行うことができるため，より効率的な管理が可能となる．ハッシュが有効に働くためには，各目次の項目から作られるリストができるだけ均一になるようにする必要がある．このため，格納する記号データの集合 D に対して，$d \in D$ から計算される値 $f(d)$ ができるだけ D を均等に分類する関数によってインデックスを作成する．このような関数を**ハッシュ関数**とよぶ．コンパイルを始める時点で D は既知ではないので，D の要素が一様に分布するというような仮定のもとに f を選択する．ハッシュ関数によってうまくデータが分散されれば，アクセスは高速になり，追加・削除に要する時間も削減できる．ハッシュが大きく偏る場合は，線形にならべた場合と同じになる．

ハッシュ関数として以下のような関数を考える．

$$f(a_1 \cdots a_n) = \left\{ \ \sum_1^n \mathsf{ord}(a_i) \cdot 63^{n-1} \ \right\} \mathsf{mod}\ N + 1$$

ここで，識別子のトークンの文字列は，_(アンダースコア), a, . . . , z, A, . . . , Z,0, . . . , 9 の系列であり，それぞれ 1 から 63 までの整数が割り当てられているとする．ord はその値を返す関数である．この計算は除算法と呼ばれる方法であり，N の値として素数を選ぶのが適当である．N が 2^k とすると f の値が下位の文字のみによって定まってしまう．

$N = 1009$ とすると，文字列 main は以下の値を持つ．

$$f(\mathtt{main}) = (14 \times 63^3 + 2 \times 63^2 + 10 \times 63 + 15)\mathsf{mod}\ 1009 + 1 = 949$$

記号 σ に対する記号表のデータは $f(\sigma)$ でアクセスするようにするが，ハッシュ関数では $f(\sigma_1) = f(\sigma_2)$ となる異なった記号 σ_1,σ_2 が存在するため，σ_1 の情報と σ_2 の情報をリストにしてアクセスする．

ハッシュを用いたインデックスづけでは，ハッシュ表に同じハッシュ値をもつ記号に対するリストの先頭を格納する．ハッシュ値 i に対するリストのことをバケット i と呼ぶ．ハッシュ表に格納される要素の数がハッシュ表のサイズの定数倍で抑えられるとき，ハッシュ表全体に一様にランダムに値がばらまかれると仮定すると，検索，登録，削除の各要素は定数時間で行うことができる．逆に，一つのバケットに格納された場合は線形リストと同じで要素の数に対する線形時間となる．このため，ハッシュ関数としてできるだけ分布が偏らない関数を用いる．

図 5.1 のコンパイルにおける記号表の構成例を以下に示す．

(1) 1 行目のコンパイル:
`int x` の宣言によって `x` を割り当てる (図 5.3a).

(2) 2 行目のコンパイル:
`int y` の宣言によって `y` を割り当てる (図 5.3b).

(3) 3 行目のコンパイル:
関数 `f` の宣言と仮引数 `int y` を割り当てる (図 5.3c).

(4) 7 行目のコンパイル:
6 行目で仮引数 y を削除して関数 main の宣言を追加する (図 5.3d).

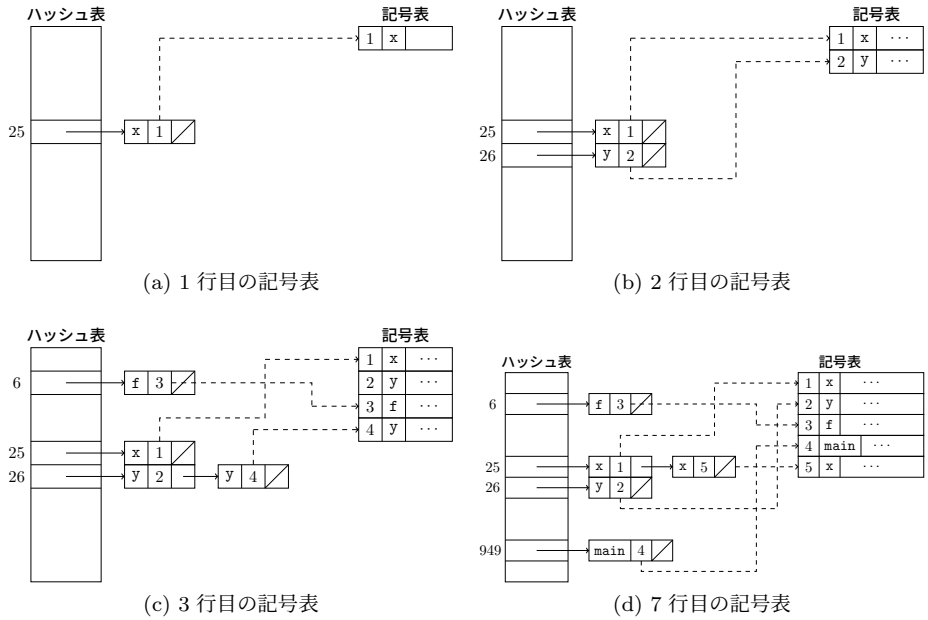


図 5.3: コンパイルの過程における記号表の構成

上記のようにハッシュ表には外部に格納された情報情報へのリンクを格納する方法を特に**外部ハッシュ**と呼ぶことがある．これに対してハッシュ表内部に情報を格納する方法は**内部ハッシュ**と呼ばれる．一般には，情報の柔軟性の高い外部ハッシュが用いられることが普通である．

5.2　型チェック

型チェックは，プログラム中のデータが整合した型を持つかどうかを検査する．コンパイル時に型チェックを行う**静的型チェック**と実行時に型チェックを行う**動的型チェック**がある．静的型チェックはプログラムを実行する前にプログラムの矛盾を検出し，効率的な実行コードを生成することが可能である．動的型チェックはより柔軟な記述を許すことができるが，型が実行時に決まるために実行時のメカニズムが複雑になる．

型チェックの手法はプログラミング言語の型付けのポリシーによる．コード生成の観点からは，型はできるだけ単純な型を持つことが望ましい．しかし，より柔軟性の高い記述のために，多相型や型のオーバーロードのような手法を用いて自然な記述を可能にすることが一般によく行われる．

再帰的な型定義を持つ場合は効率的にコンパイルするためには 2 パスで型チェックを行う必要であることもある．変数の型，関数，メソッドの返り値と引数の型が同じ識別子ですべて同じであるかを構文解析時にチェックすることで，プログラムの不整合を事前に検出する．

静的な型チェックとして，例えば，加算式 $e_1 + e_2$ をコンパイルするとき，e_1，e_2 の型によって内部狂言が異なるため，生成する命令が異なる．両辺が整数型の場合は，整数の加算命令を発行

し，片方が整数で，他方が実数である場合は，整数を実数の型に変換する命令を発行してから実数の加算命令を発行する．ML のように型によって異なる演算記号をもち，型変換を行わずにエラーとする言語もある．また，文字列などのように加算することができない型である場合は，エラーを発行したり，結合など別の操作のためのコードを発行する．

以下ではコンパイラ時に型の整合性を検査する静的型チェックを中心に説明する．

5.2.1 型定義と型チェックアルゴリズム

簡単な型チェック法を次の生成規則で定義される言語を考える．

$$
\begin{aligned}
&P \to D \ ; \ S \\
&D \to D \ ; \ D \mid \mathbf{id} : \ T \\
&T \to \texttt{char} \mid \texttt{int} \mid \texttt{\&}T \mid \texttt{arr[}n\texttt{]} \ \texttt{of} \ T \\
&E \to c \mid n \mid \mathbf{id} \mid E \ \texttt{+} \ E \mid \texttt{*}E \mid E\texttt{[}E\texttt{]} \\
&S \to \mathbf{id}\texttt{:=}E \mid S\,;S
\end{aligned}
$$

プログラムは，変数の型宣言部 D とプログラム本体 E からなる．E はここでは簡単のため加算と配列参照のみからなる式としている．i : &T は，変数 i が型 T を持つデータ領域へのポインタであることを示す．$*i$ は i が型 T を持つデータ領域へのポインタであるとき，i の指す型 T を持つデータ領域を表す．型に関する制約は，加算 + は整数に対してのみ可能であり，$E[E']$ においては，E の型は配列型 (arr[n] of T) であり，E' の型は整数型 (integer) である．このとき $E[E']$ の型は T である．図 5.4 に宣言における型定義の規則，図 5.5 に型制約規則を示す．

生成規則	型規則
$P \to D$; E	
$D \to D$; D	
$D \to i$: T	i.type:=T.type (記号表に登録)
$T \to$ char	T.type:=char
$T \to$ int	T.type:=integer
$T \to$ &T_1	T.type:=pointer(T_1.type)
$T \to$ arr[n] of T	T.type:=array($1..n$.value, T_1.type)

図 5.4: 型定義の規則

生成規則	型制約規則
$E \to c$	E.type:=char
$E \to n$	E.type:=integer
$E \to i$	E.type:=i.type (記号表を検索する)
$E \to E_1$+E_2	E.type:= **if** E_1.type = integer **and** E_2.type = integer **then** integer **else** type_error
$E \to$ *E_1	E.type:= **if** E_1.type = pointer(t) **then** t **else** type_error
$E \to E_1$ [E_2]	E.type:= **if** E_1.type = array(s,t) **and** E_2.type = integer **else** type_error
$S \to i$:=E	i.type= E.type

図 5.5: 型制約規則

c は文字トークン，n は整数定数のトークンを表す.

```
char x;
int y;
&int z;
arr[10] of char a;
arr[5] of arr[5] of int m;
```

ここで，T.type は型識別子 T が表す型であり，通常，T の表現そのものである．i.type は識別子 i が持つ型情報であり，記号表に記録される．n.value は，数トークン n が表す値であり，n の表現そのもの $(0, 1, \cdots$ など) である．型制約のない場合，上記のように宣言されているとき，`y:=*z`, `a[y]` は型づけとして正しい (well-typed)．`x:=1`, `x+y`，`x+1`，`m[x]` は型エラーである．

型制約は構文解析に沿って検査可能である．下向き構文解析の場合は，規則の最後で上のレベルに戻る際に一致をチェックする．上向き構文解析の場合は還元の際に属性として検査することができる．

5.2.2　多相型と型のオーバーロード

原則的には構文的に対応する場所には同じ型がなければならない．直観的な値に対する型の概念は，その値に対してどういった操作を適用できるかということで決定される．したがって，型が厳密に一致していなくても互換性のある操作であれば認める方がプログラムとしては記述しやすい．型の一致については，多層型やオーバーロードという方法で矛盾することなく型の概念が適用できる．例えば，整数に対しては，四則演算を適用できる値の集まりであると特徴づけられる．これに対して，文字列については，文字を表す記号列に対して，連接や接頭語，接尾語などを取り出す操作をもつ値であり，整数型とは異なる．

多相型

多相型 (polymorphic type) は，操作が共通した型を表す．リストや配列は，同じ型を持つ値から構成されるデータ型である．リストに対しては，要素の型にかかわらず連接，空チェックなど共通

の操作が定義できる．List(X) のように記述し，変数 X に要素データの型を代入し，List⟨Int⟩, List⟨Char⟩ のように記述することによって同じ操作を持つデータ型を記載する．このことによって操作を要素ごとに重複して記載する必要がない．

多相型を持つプログラミング言語の記述として，Java，Ada や Modula-3 における *generic* 型，C++におけるテンプレート，ML におけるタイプスキームなどがある．

部分型

型 A に対する操作が型 B に対しても適用できるとき，型 B は型 A の部分型であるとよばれる．部分型はもとの型を拡張することによって定義される．多相型 $T\langle X\rangle$ を構成する場合，型 B が型 A の部分型であるとき，$T\langle A\rangle$ が矛盾のない型であるとき，$T\langle B\rangle$ も矛盾のない型として定義される必要がある．

オーバーロード

```
1                int x;
2                float y;
3                double z;
4                x=1; y=1.1; z=1.57e8;
5                x=x+1;
6                z=y+z;
7                z=z+x;
```

オーバーロードは同じ種類の操作（通常同じ記号で表す）を引数の型によって切り替える機能である．多くの言語は四則演算において，オーバーロード機能を持つ．

左のような加算は型の異なる数に対して加算を実行する．5,6,7 行目の演算はすべて加算であるが，変数 x,y,z の内部表現はすべて異なるのでコンパイラが発行する命令は異なる．計算機の実行上では，5,6,7 行目は異なる命令を発行する必要がある．オーバーロードがない場合，同じ加算であっても型が異なる場合は異なる演算記号を用いる必要がある．オーバーロードがない関数型言語 ocaml では整数の加算は+, 浮動小数点の加算は+. を用いる．